ENGINEERING FOR AI SYSTEMS

AI Engineer Portfolio

AI를 모델 호출에서 끝내지 않고, 데이터·추론·API·인터페이스가 이어지는 서비스로 구현합니다.

핵심 포지셔닝

Python 기반 AI 시스템을 직접 설계하고, 모델 처리 흐름을 백엔드 API와 사용자 인터페이스까지 연결합니다.

SELECTED PROJECTS

01 / PROJECT

Codebase AI Assistant

RAG로 Python 코드베이스를 검색하고, 파일 경로를 포함한 답변을 생성합니다.

Python · FastAPI · ChromaDB · Ollama

02 / PROJECT

AI Smart Surveillance Platform

YOLO11 탐지와 ByteTrack 추적을 제한 구역 이벤트 및 웹 대시보드로 연결합니다.

YOLO11 · OpenCV · FastAPI · React

ENGINEERING CAPABILITIES

01 / AI PIPELINES	02 / SERVICE INTEGRATION	03 / DEBUG & ITERATE
RAG 검색, 임베딩, 로컬 LLM 추론, 컴퓨터 비전과 객체 추적 흐름을 구현합니다.	FastAPI를 통해 AI 처리 결과를 React·TypeScript 인터페이스에 연결합니다.	구현 한계와 입력·상태 흐름을 확인하고, 다음 개선 항목을 구분합니다.

WORKING METHOD

문제 정의	시스템 설계	구현	검증	개선

CONTACT / PROFILE

EMAIL **soundaim56@gmail.com**

GITHUB **github.com/Hyuki56**

PROJECT 01 / LLM · RAG

Codebase AI Assistant

코드 구조를 검색 가능한 근거로 바꾸는 로컬 RAG 서비스

문제 정의

코드가 파일과 함수에 분산되어 있으면 동작의 위치와 맥락을 찾아 이해하기 어렵습니다. 저장소 전체를 LLM에 전달하는 방식은 입력 문맥과 출처 확인에 제약이 있습니다.

END-TO-END FLOW

Python 소스 수집	AST 기반 청킹	임베딩 · ChromaDB	검색 · Ollama · API

주요 구현

구조 단위 코드 분할

Python AST로 최상위 클래스와 함수를 분할하고, 긴 조각은 최대 1,500자 기준으로 안전하게 다시 나눕니다. 파싱 실패 파일은 텍스트 분할로 처리합니다.

검색 출처 보존

파일 경로, 조각 유형, 이름을 검색 메타데이터로 유지해 답변 문맥과 원본 코드를 연결합니다.

모델과 API

nomic-embed-text와 qwen2.5-coder:7b를 Ollama에서 사용하고 색인·질의 기능을 FastAPI로 제공합니다.

기술 선택

ChromaDB로 코드 임베딩을 저장하고 유사 코드 검색에 활용합니다. 로컬 모델 구성을 통해 외부 유료 AI API 없이 실행합니다.

문제 해결 경험

문제

긴 코드 조각이 모델 입력 문맥을 과도하게 차지할 수 있고, 응답만으로는 근거 파일을 찾기 어렵습니다.

대응

청크 길이를 제한하고 실패 파일은 텍스트 기반으로 분할했습니다. 검색 메타데이터에 파일과 구조 정보를 보존해 출처를 응답에 포함했습니다.

기술 스택

Python · FastAPI · ChromaDB · Ollama · nomic-embed-text · qwen2.5-coder:7b

한계와 다음 단계

중첩 심볼·함수 관계 모델링, 검색 품질 평가, 증분 색인은 추가 개선 항목입니다. 정확도나 검색 성능 수치는 측정 결과가 없어 기재하지 않았습니다.

실행 흐름과 시각 근거는 4페이지에서 확인

GITHUB REPOSITORY github.com/Hyuki56/codebase-ai-assistant ->

PROJECT 02 / COMPUTER VISION · FULL-STACK

AI Smart Surveillance Platform

프레임의 탐지를 제한 구역 이벤트와 웹 모니터링으로 연결

문제 정의

사람 탐지만으로는 제한 구역 진입 여부나 상황 기록을 해결할 수 없습니다. 추적 ID, 구역 상태, 이벤트 저장, API와 대시보드를 하나의 흐름으로 연결했습니다.

VIDEO-TO-EVENT FLOW

영상 · YOLO11	ByteTrack 추적	발 위치 · Polygon	ENTER/EXIT · SQLite

주요 구현

사람 탐지와 추적

YOLO11로 사람을 탐지하고 ByteTrack으로 프레임 간 tracker ID를 유지합니다. OpenCV로 프레임 처리와 화면 표시를 구성했습니다.

구역 판정과 이벤트

바운딩 박스 하단 중앙인 발 위치를 Polygon 구역과 비교합니다. tracker ID별 상태 변화로 ENTER/EXIT를 구분하고 SQLite 이벤트 및 진입 스냅샷을 저장합니다.

API와 인터페이스

FastAPI가 상태·이벤트·스냅샷 API와 MJPEG 스트림, 감시 시작·중지 기능을 제공합니다. React·TypeScript 대시보드가 이를 연결합니다.

기술 선택

탐지 결과를 추적 ID와 구역 판정에 연결해 이벤트 흐름으로 구성했습니다. 화면과 API에서 상태를 확인하고 제어할 수 있습니다.

문제 해결 경험

구역 판정 기준

박스 중심은 사람의 바닥 접점을 나타내기 어렵습니다. 하단 중앙 foot point를 Polygon 포함 판정에 사용했습니다.

스냅샷 맥락

이벤트 순간의 상황을 검토할 수 있도록 프레임에 박스·ID·구역 상태를 표시한 뒤 ENTER 시점 이미지를 저장합니다.

기술 스택

Python · YOLO11 · OpenCV · ByteTrack · FastAPI · React · TypeScript · SQLite

한계와 다음 단계

영상 경로와 구역 좌표는 코드에 고정되어 있습니다. 다양한 환경의 탐지 평가, 장시간 실행 안정성, 설정 UI, 원격 접근 제어를 개선할 수 있습니다.

실행 흐름과 시각 근거는 4페이지에서 확인

GITHUB REPOSITORY github.com/Hyuki56/AI-Smart-Surveillance-Platform ->

실행 근거와 데이터 흐름

원본 프로젝트 코드와 저장된 감지 프레임을 바탕으로 정리했습니다.

01 / CODEBASE AI ASSISTANT · RAG PIPELINE

01 SOURCE	02 CHUNK	03 INDEX	04 STORE
Python 저장소	AST 구조 분할	Ollama 임베딩	ChromaDB

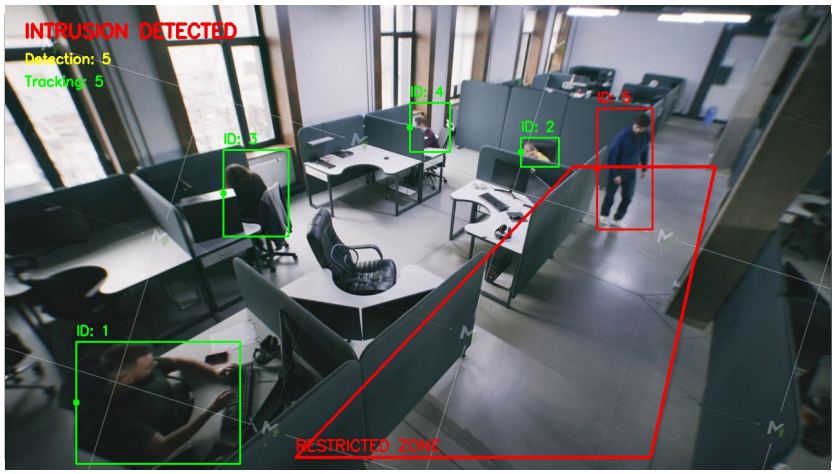
질의 흐름

질문 임베딩 → 유사 코드 검색 → 경로·청크 정보를 포함한 문맥 구성 → qwen2.5-coder:7b 답변

API: POST /api/index · POST /api/ask · 응답에 검색 출처 파일 경로 포함

구현상 한계: 색인 요청 시 기존 ChromaDB를 초기화하고 저장소 전체를 다시 색인합니다.

02 / AI SMART SURVEILLANCE · ACTUAL DETECTION FRAME



실제 저장된 감지 스냅샷 · 탐지 박스, tracker ID, 감시 구역과 침입 상태

판정과 이벤트

YOLO11 사람 탐지 →
ByteTrack ID 추적 →
바운딩 박스 하단 중앙을
foot point로 사용 →
OpenCV Polygon 판정

OUTSIDE → INSIDE
ENTER · 시각·스냅샷 저장

INSIDE → OUTSIDE
EXIT · 체류 시간 업데이트

서비스 연동

FastAPI	상태 · 시작/중지 · 이벤트 · 스냅샷 · MJPEG
React + TypeScript	영상 모니터링과 감시 제어 UI

API: GET /status · GET /events · GET /video-feed · POST /start · POST /stop

현재 한계: 영상 경로·감시 구역 좌표가 코드에 고정되어 있고, 환경별 정량 평가는 추가 과제입니다.